

Irembo Voice AI Intent Classification System Report

18th January 2026

GitHub Repository: <https://github.com/Abdulraqib20/irembo-voice-ai>

Overview: I implemented and evaluated a multilingual intent classifier for Irembo Voice AI, centered on AfroXLMR fine-tuning with a confidence-based fallback pipeline. The repository includes the full data preprocessing, training, evaluation, and API inference workflow used to generate the results in this report.

Approach

I fine-tuned a multilingual transformer, Davlan/afro-xlmr-base, for intent classification. This model was selected because it includes African language coverage and handles morphology and code-switching between Kinyarwanda and English. In production, I use a hybrid routing strategy: the transformer is primary, a rule-based classifier provides a fast baseline and secondary fallback, and a Mixtral-8x7B LLM fallback is used for low-confidence or ambiguous cases. The transformer remains the primary model because it delivers higher macro performance across all intents.

Why this fits Voice AI: Voice AI systems face unique NLP constraints that guided model selection:

- **ASR noise tolerance:** Speech-to-text introduces errors (“rendez-vous” → “rendezvous”, “pasiporo” → “passport”). Transformers learn robust representations that generalize across spelling variants, unlike brittle keyword matching.
- **Code-switching:** Rwandan users naturally mix Kinyarwanda and English (“Ndashaka kureba status”). AfroXLMR’s multilingual pretraining handles cross-lingual subword sharing.
- **Informal phrasing:** Spoken language differs from written text including contractions, hesitations, incomplete sentences. The model sees diverse training examples including informal patterns.
- **Latency requirements:** Voice interactions expect sub-second responses. The transformer runs in 60–100ms on CPU, acceptable for real-time use.

Data Strategy

Cleaning noisy voice transcripts I prioritized **minimal normalization**, strictly limiting preprocessing to whitespace collapse and schema validation. I retained ASR artifacts (such as phonetic approximations) to ensure the training data matches the noisy input expected from the production ASR, preventing a “clean train / dirty test” mismatch that often degrades real-world performance.

Handling code-switching The dataset contains Kinyarwanda (rw), English (en), and mixed utterances. I preserve code-switched tokens intact—no language-specific filtering. AfroXLMR’s subword tokenizer learns shared representations across languages. A lightweight language detector (`src/services/enhanced_language_detector.py`) scores utterances using marker words (e.g., “ndashaka”, “igeze” for rw; “status”, “application” for en) and flags mixed when both scores >0.

Handling code-switching Given the dataset contains Kinyarwanda (rw), English (en), and mixed utterances, I adopted a strategy that preserves code-switched sequences intact without language-specific filtering. AfroXLMR’s subword tokenizer learns shared representations across languages. A lightweight custom language detector scores utterances using marker words (e.g., “ndashaka”, “igeze” for rw; “status”, “application” for en) and flags mixed when both scores >0.

Small and imbalanced data Faced with a limited and imbalanced dataset of 700 samples (561 train, 70 val, 69 test) across 13 intents. I track intent/language distributions in preprocessing and use **Macro F1-Score** (not accuracy) to weight rare intents equally. The **Hybrid Fallback mechanism** (rule-based + LLM) reduces errors on underrepresented cases while new data accumulates.

Data quality improvement over time I established an **Active Learning loop** driven by production telemetry. The system automatically flags predictions with confidence scores below 0.7 for human review and relabeling. This creates a continuous feedback cycle to retrain the model on its hardest edge cases, with all iterations tracked in a versioned registry.

Evaluation

I report accuracy, Macro Precision, Macro Recall, and Macro F1. Macro metrics are chosen to weight all intents equally, which is critical for public services. Weighted metrics can hide poor performance on rare intents.

Model	Accuracy	Macro F1	Macro Precision	Macro Recall
Rule-Based Baseline	88.41%	89.37%	90.34%	88.57%
AfroXLMR base	95.65%	95.63%	96.54%	95.26%

Table 1: Overall test performance.

Per-language accuracy English 90.5%, code-switched 100.0%, Kinyarwanda 97.4%. This demonstrates strong performance on the target language and on mixed utterances.

Robustness and edge-case testing I test robustness with the transformer, baseline, and hybrid evaluation scripts and also probe the live API. Empty inputs are rejected by request validation, ambiguous or mixed-language text is routed through confidence-based fallback, and long utterances are safely handled by the configured max sequence length (128).

Common failure modes Failure modes are identified from the hybrid evaluation confusion matrix and sample errors, and they are logged at inference with explicit fallback reasons. The main patterns are confusable intents (e.g., requirements vs fees) and out-of-scope queries that drive low confidence; when primary or fallback classifiers fail, **unknown** is returned and the event is recorded.

Suitability for a citizen-facing system Suitability is assessed via macro F1, per-language accuracy, and operational safeguards in the API. Each request is logged with confidence, fallback reason, and latency; monitoring tracks low-confidence rate, fallback rate, P95 latency, and language drift. Health/readiness endpoints support safe rollout, and escalation is available when confidence drops below the defined threshold.

MLOps and Production Considerations

Model Versioning and Deployment

I use a simple model registry to track versions, training configuration, evaluation metrics, and the active model with rollback support. The model is deployed behind a FastAPI service with health/readiness checks and both single and batch inference routes. Docker Containerization is used for reproducible deployment and predictable runtime behavior.

Monitoring in Production

Inference is logged per request with confidence, latency, language, and fallback reason, and metrics are aggregated over a rolling window. Drift is monitored via language distribution shifts; performance is tracked with low-confidence and fallback rates plus P95 latency. Alerts are triggered when thresholds are exceeded, and logs are retained for offline error analysis.

Real-Time Voice AI Pipeline Integration

The pipeline is integrated as: ASR produces text, language is detected, the transformer predicts intent with confidence, and low-confidence cases are routed through fallback. Predicted intents are then mapped to downstream service actions. The real-time API supports both single and batch classification for online and offline flows.

Fallback Strategies

I use a three-tier fallback: transformer first, rule-based second, and an LLM (Mixtral-8x7B) as a last resort for ambiguous cases. Fallback is triggered by confidence thresholds and every fallback decision is logged for review. This balances accuracy, latency, and interpretability in a citizen-facing setting.

Voice AI Pipeline Architecture

The implemented flow:

Utterance Text → **Language Detection** (en/rw/mixed) → **Intent Classification** (Transformer → Rule-based → Mixtral-8x7B (LLM) fallback) → **Response**.

Each request is logged with confidence, fallback reason, and latency for observability.

Ethics and Public Sector Considerations

Bias and fairness risks Government services must serve all citizens equitably. Key risks and mitigations:

- **Language Bias:** Kinyarwanda speakers should not receive worse service than English speakers. I evaluate per-language accuracy separately (Kinyarwanda 97.4% vs English 90.5%) and prioritize Kinyarwanda performance.
- **Intent Coverage Bias:** Rare intents (e.g., `complaint_or_support_ticket`) may be under-represented. Macro F1 weighs all intents equally, preventing majority-class dominance.

Explainability: Confidence scores and fallback reasons are returned and logged, and low-confidence cases are escalated. Intent definitions include canonical examples to support transparent responses.

Responsible AI: Uncertain predictions are routed through the fallback tiers, and an escalation signal is available when confidence drops below threshold. Each request is logged with a request ID and fallback reason for auditability, and monitoring flags degradation via low-confidence, fallback-rate, latency, and language-drift alerts.

Tradeoffs and Limitations

- **Accuracy vs. latency:** Transformer (95.65% acc, ~60–100ms) vs. rule-based (88.4% acc, ~16ms). I prioritized the transformer’s higher accuracy, with the rule-based kept as a fallback.
- **Model size vs. deployment cost:** AfroXLMR-base (278M params) is heavier than a small model; a lighter or distilled model would reduce resource needs at some accuracy cost.
- **External API dependency:** Mixtral-8x7B LLM (via Groq) fallback adds cost (\$0.05–0.10 per 1K tokens) and latency (~200–400ms). For cost-sensitive deployments, disable LLM tier or use local smaller LLM.
- **Dataset size:** 700 samples is small, results should be validated with real prod traffic, and expected to improve with production feedback.

Multilingual expansion strategy: To extend beyond Kinyarwanda/English:

1. **Add language data:** Collect labeled utterances in target language (e.g., French, Swahili).
2. **Update detection:** Extend the lightweight language marker list to recognize the new language.
3. **Fine-tune and evaluate:** Continue training from the current checkpoint using a mixed dataset, then report per-language metrics to ensure the new language does not regress existing performance.

Conclusion

AfroXLMR-base provides a reliable multilingual intent classifier for the Irembo Voice AI use case. Strong performance is achieved on Kinyarwanda (97.4%) and code-switched speech (100%), and the system integrates with confidence-based fallbacks (rule-based + Mixtral-8x7B LLM) and comprehensive monitoring. It is designed to be fair (Macro F1), transparent (decision logging), and resilient (three-tier fallback) for a citizen-facing platform.

A working demo is available via Dockerized FastAPI endpoints with health checks, batch processing, and real-time classification.